

Лабораторная работа №1. Основы Kotlin и работа в IntelliJ IDEA

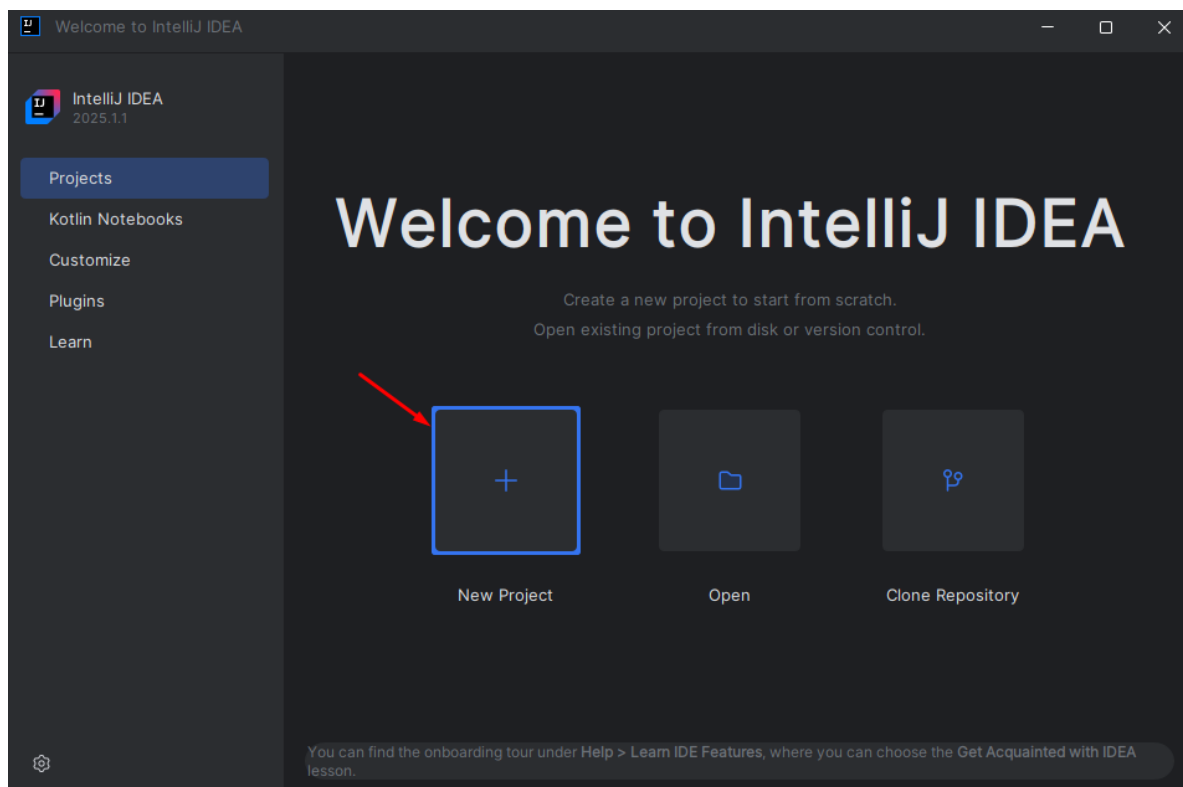
Цель: Познакомиться с языком Kotlin и базовыми возможностями среды разработки IntelliJ IDEA, научиться создавать простейшие программы, работать с переменными, типами данных и математическими операциями.

Задачи:

- Освоить создание Kotlin-проекта в IntelliJ IDEA.
- Изучить стиль кодирования и оформление комментариев в Kotlin.
- Освоить работу с переменными (val, var) и шаблонами строк.
- Понять базовые типы данных, преобразования и арифметику.
- Освоить консольный ввод и вывод, работу с ошибками в IDE.
- Изучить поведение инкремента, декремента, целочисленного и дробного деления.

Шаг 1. Создание проекта в IntelliJ IDEA

1. Запустите **IntelliJ IDEA**. Если у вас еще нет созданных проектов, на стартовом экране выберите **New Project**:



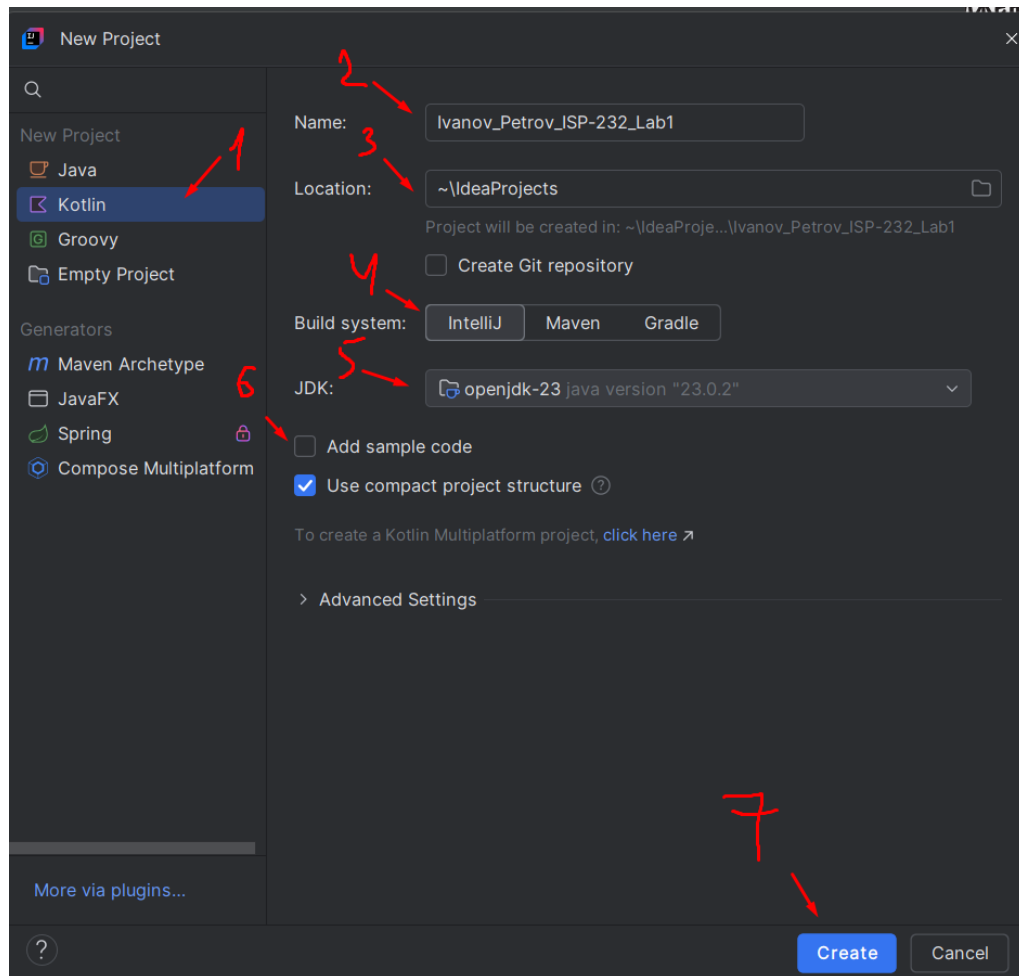
2. Откроется окно создания проекта. Настройте его следующим образом:

1 – **Выберите язык:** Kotlin.

2 – **Название проекта:** введите в формате: *Фамилия_Имя_Группа_Lab1*
Ivanov_Sidorov_ISP-231_Lab1

3 – **Location**: путь к проекту можно оставить по умолчанию или задать вручную.

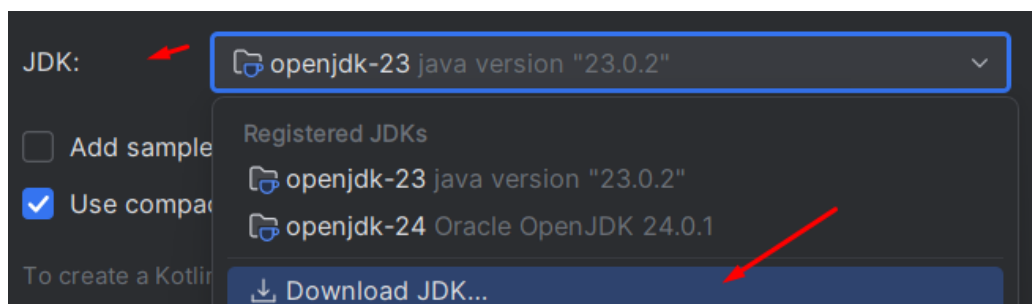
4 – **Build system**: оставляем **IntelliJ**.



Кратко о системах сборки:

- **IntelliJ IDEA** — стандартная среда, упрощает настройку, подходит для начальных проектов.
- **Maven** — инструмент с XML-конфигурацией, часто используется в Java-проектах.
- **Gradle** — гибкий инструмент с Kotlin/Groovy-синтаксисом, применяется в Android-разработке.

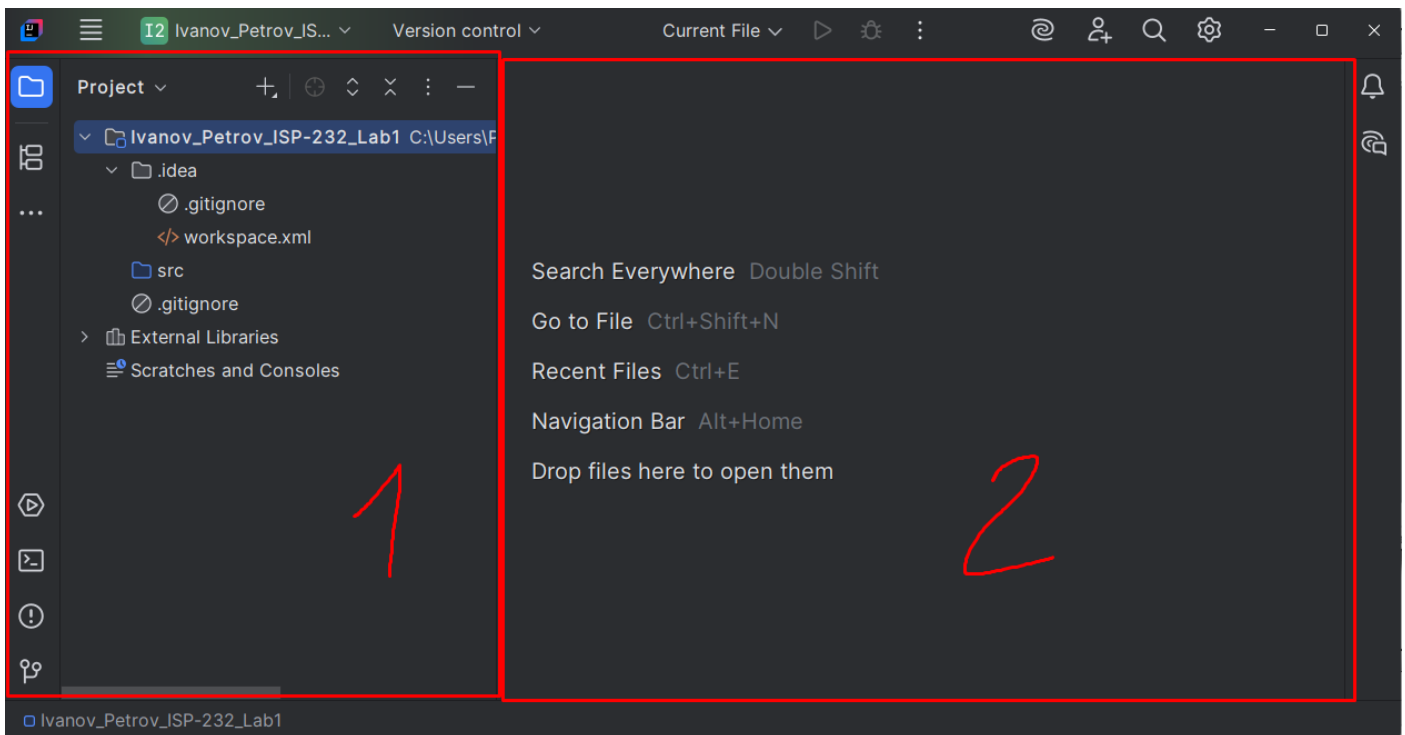
5 – **JDK**: выберите установленную версию JDK (если поле пустое — укажите путь вручную). Обычно IntelliJ автоматически находит установленную JDK. Если это поле пусто, то его надо установить:



6 – **Add sample code**: отключите эту опцию (уберите галочку), чтобы начать с пустого проекта.

7 – Нажмите **Create** — проект будет создан.

3. Когда проект откроется:



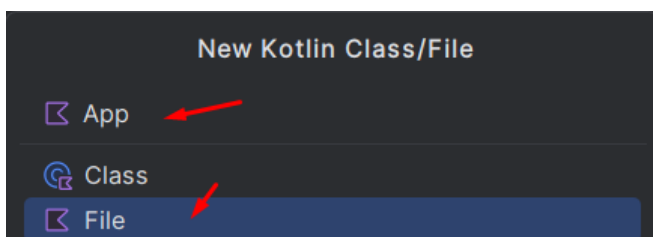
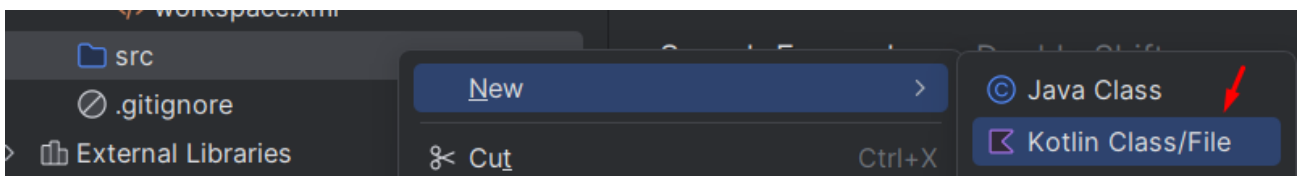
1. Project View (слева) — структура файлов проекта.

2. Editor (центр) — редактирование выбранных файлов.

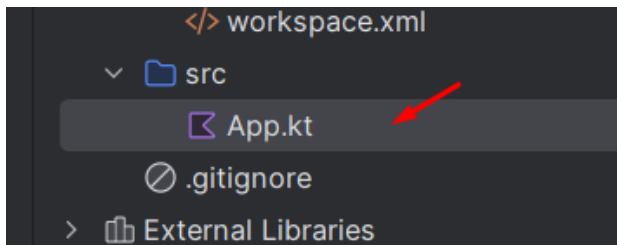
Исходный код размещается в папке `src`, которая пока пустая. Добавим файл вручную.

Все файлы с исходным кодом помещаются в папку **src**. По умолчанию она пуста, и никаких файлов кода у нас в проекте пока нет. Поэтому добавим файл с исходным кодом.

- Кликните правой кнопкой по папке **src**.
- Выберите: **New** → **Kotlin Class/File**
- Введите имя: **App**
- Выберите тип: **File**



Теперь у вас должен появиться файл **App.kt** внутри папки **src**:

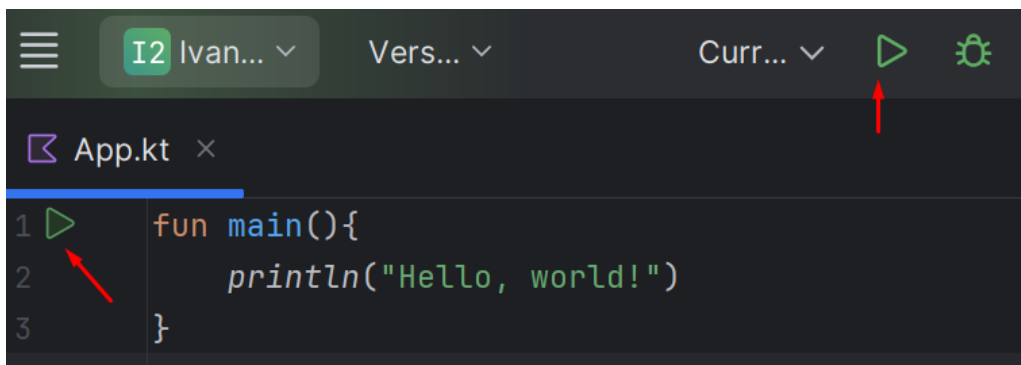


Также наш файл открылся, но по умолчанию он пустой. Давайте напишем в нём простой код, который будет выводить “**Hello, world!**”:

```
1 fun main(){
2     println("Hello, world!")
3 }
```

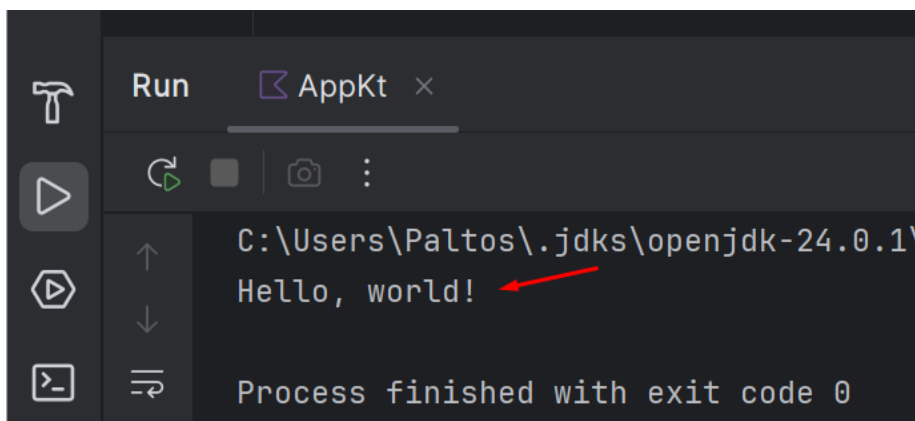
Чтобы запустить код:

- Откройте меню **Run** → **Run 'AppKt'**
- Или нажмите **Shift + F10**



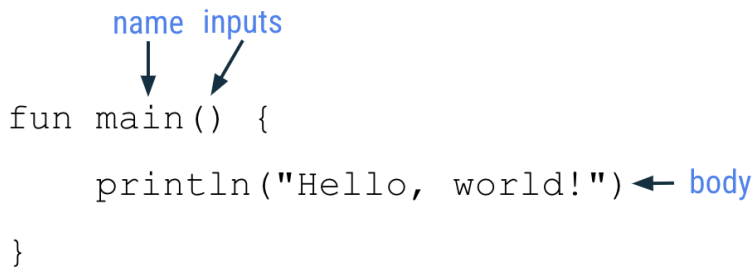
IDE компилирует исходный код Kotlin в байт-код JVM. Затем IDE запускает JVM и выполняет класс App.kt.

После выполнения внизу окна вы увидите вывод:
Hello, world!



Готово! Вы создали свой первый проект и написали простую программу на языке **Kotlin**.

Обратите внимание на ключевые части функции в main примере функции:



```

name inputs
  ↓   ↓
fun main() {
    println("Hello, world!") ← body
}

```

- Определение функции начинается со слова **fun**.
- Тогда имя функции будет **main**.
- Входных данных для функции нет, поэтому скобки пустые.
- В теле функции есть одна строка кода, `println ("Hello, world!")` которая расположена между открывающей и закрывающей фигурными скобками функции.

Теория. Руководство по Kotlin style

Одной из хороших практик кодирования является следование стандартам. Мы кратко рассмотрим стандарты кодирования Android от Google для кода, написанного на Kotlin.

- Имена функций должны быть написаны в верблюжьем стиле и представлять собой глаголы или глагольные фразы.
- Каждое утверждение должно располагаться на отдельной строке.
- Открывающая фигурная скобка должна находиться в конце строки, где начинается функция.
- Перед открывающейся фигурной скобкой должен быть пробел.

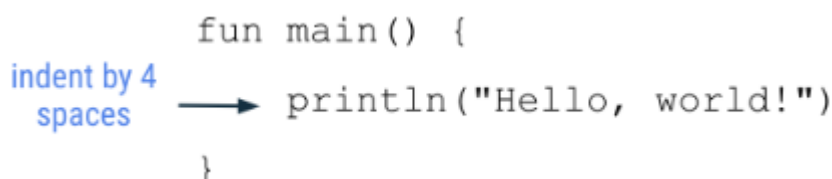


```

space
  ↓
fun main() {
    println("Hello, world!")
}

```

- Тело функции должно иметь отступ в 4 пробела. Не используйте символ табуляции для отступа кода, введите 4 пробела.



```

      fun main() {
indent by 4 spaces → println("Hello, world!")
      }

```

- Закрывающая фигурная скобка находится на своей собственной строке после последней строки кода в теле функции. Закрывающая фигурная скобка должна быть выровнена с fun ключевым словом в начале функции.

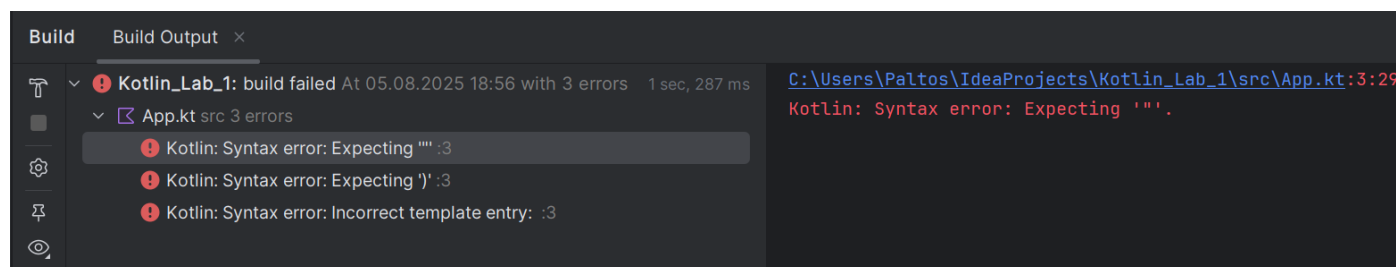
```
aligned  
vertically  
fun main() {  
    println("Hello, world!")  
}
```

Шаг 2. Исправление ошибки в IDE

1. Скопируйте и вставьте следующий фрагмент код и запустите программу.

```
fun main() {  
    print("Hello world!")  
    println("Today is sunny!")  
}
```

2. В идеале вы хотите увидеть отображаемое сообщение Today is sunny!. Вместо этого в панели вывода вы видите значки восклицательных знаков с сообщениями об ошибках:



Сообщения об ошибках начинаются со слова "Exprecting", потому что компилятор Kotlin "ожидает" чего-то, но не находит этого в коде. В этом случае компилятор ожидает закрывающую кавычку и закрывающую скобку для кода во второй строке вашей программы.

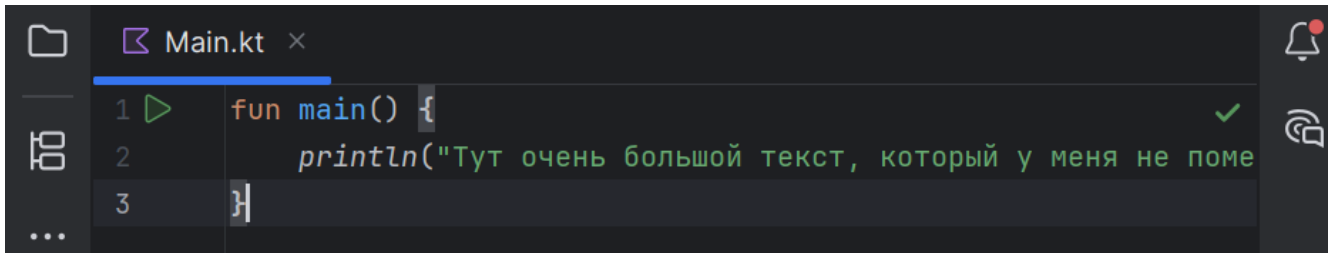
3. Добавьте закрывающую кавычку после восклицательного знака перед закрывающей скобкой.

```
fun main() {  
    println("Today is sunny!")  
}
```



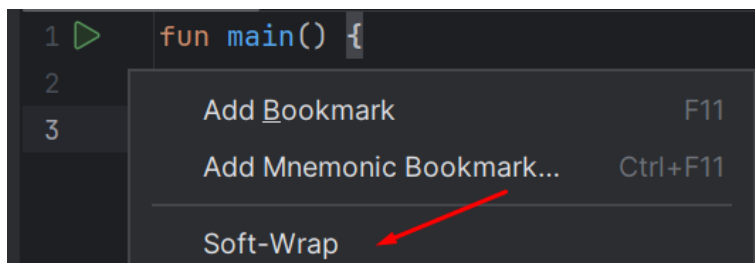
Шаг 3. Настройка переноса текста (Soft-Wrap) в IntelliJ IDEA

При работе с длинными строками вы можете столкнуться с тем, что текст выходит за границы экрана:



1. Быстрое включение переноса:

1. Наведите курсор на левый край редактора кода
2. Щелкните правой кнопкой мыши (ПКМ)
3. В контекстном меню выберите "Soft-Wrap"

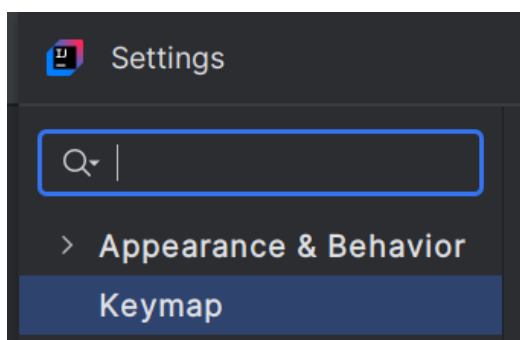


2. Назначение горячих клавиш (если функция используется часто):

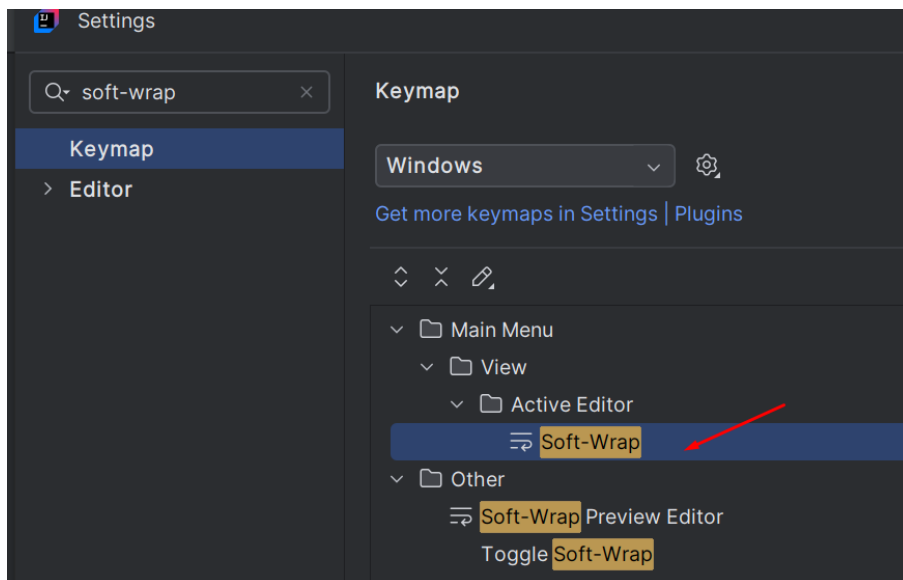
По умолчанию в IntelliJ IDEA не назначены горячие клавиши для этой функции.

Чтобы это исправить:

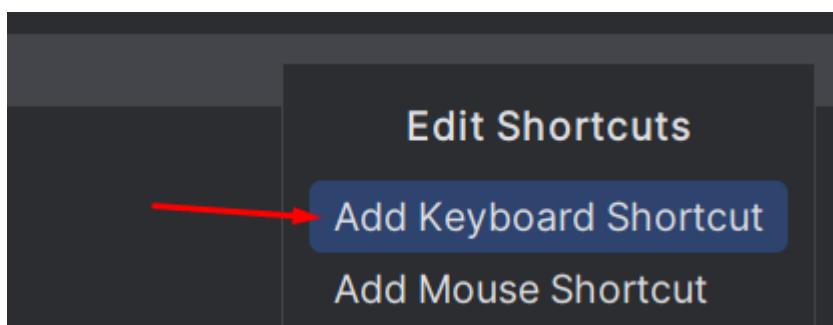
1. Перейдите в **File** → **Settings** (или **Ctrl+Alt+S**)
2. В открывшемся окне выберите **Keymap**



3. В строке поиска введите "**soft-wrap**"
4. Найдите пункт "**Soft-Wrap**" в списке команд
5. Дважды щелкните по нему

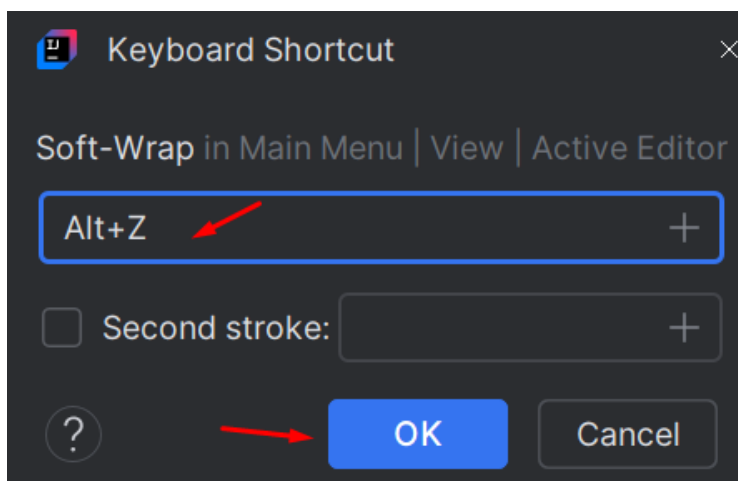


6. Выберите "Add Keyboard Shortcut"



7. Нажмите сочетание клавиш **Alt+Z**

8. Подтвердите выбор, нажав **OK**:



Шаг 4. Переменные: **val** и **var**

В Kotlin переменные бывают двух видов:

- **val** — неизменяемая переменная.
- **var** — изменяемая переменная.

Напишите следующий код:


```
fun main(){
    val city = "Москва" // Нельзя изменить после присваивания
    var temperature = 20 // Можно менять значение
    temperature += 5
}
```

📌 **Константы:** Если переменная не зависит от входных данных и должна быть

видна во всём модуле:

```
const val PI = 3.1415 1 Usage
fun main(){
    println(PI)
}
```

Особенности констант:

- Нельзя изменить значение константы
- Константа должна объявляться на самом верхнем уровне (вне класса/функции);
- Тип данных константы должен соответствовать одному из примитивных (например, Int) или типу String

📄 **Задание:**

- Объявите переменные имя, год рождения и возраст (с помощью val и var).
- Измените значение возраста (например, прибавьте 10 лет).
- Выведите значения в консоль.

Шаг 5. Базовые типы данных

В Kotlin все переменные имеют строго определенный тип.

1. Замените существующий код следующей программой:

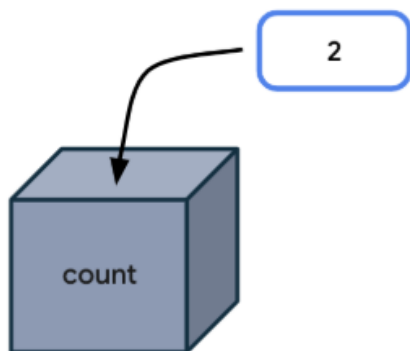
```
fun main() {
    val count: Int = 2
    println(count)
}
```

Эта программа создает переменную **count** с именем и начальным значением 2 и использует ее, выводя значение переменной **count** на выход.

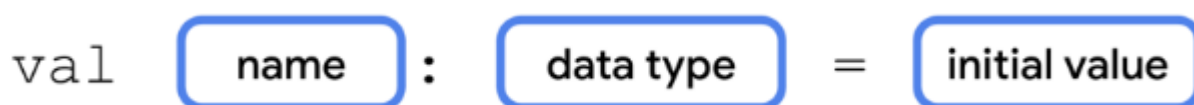
Запустите программу, и вывод должен быть следующим:

```
2
```

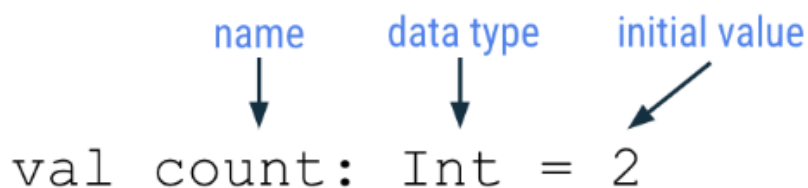
Согласно второй строке ***val count: Int = 2*** создается целочисленная переменная с именем **count**, которая содержит число 2:



Следующая диаграмма показывает, где должна располагаться каждая деталь о переменной, а также расположение пробелов и символов:



В контексте count примера переменной вы можете видеть, что объявление переменной начинается со слова **val**. Имя переменной — **count**. Тип данных — **Int**, а начальное значение — **2**:



Основные типы данных:

- **Int, Long** — целые числа
- **Double, Float** — числа с плавающей точкой
- **Boolean** — логический тип (true / false)
- **Char** — символ (один символ в кавычках 'A')
- **String** — строка
- **Any** — универсальный базовый тип (аналог Object в Java и C#)

📌 Рекомендация:

Для более глубокого понимания изучите [официальную документацию Kotlin](#).



Задание: Объявите переменные каждого из перечисленных типов, присвойте

им значения и выведите на консоль.

Шаг 6. Шаблоны строк

Шаблоны строк (string templates) представляют удобный способ вставки в строку различных значений, в частности, значений переменных. Так, с помощью знака доллара \$ мы можем вводить в строку значения различных переменных:

"string contents \$ **variable name** rest of string"

Перепишите следующий код и запустите:

```
fun main() {  
    val game = "Dota 2"  
    val year = 2005  
    val message = "$game - пользовательская карта для Warcraft III,  
        разрабатываемая с $year года"  
    println(message)  
}
```

Шаг 7. Комментарии и упражнение для мозгов

В Kotlin есть два типа комментариев:

- **Однострочный** — начинается с // и действует до конца строки.
- **Многострочный** — обрамляется /* ... */ и может занимать несколько строк.

Пример использования комментариев (напишите его в программе):

```
/*  
    многострочный комментарий  
    Функция main -  
    точка входа в программу  
*/  
fun main(){ // начало функции main  
    println("Hello, world!") // вывод строки на консоль  
} // конец функции main
```



Задание: Перепишите код ниже, закомментировав каждую строку, и попробуйте объяснить, что она делает:

```

fun main(){
    val name = "Denis" // Объявить переменную с именем
                        'name' и присвоить ей значение "Denis"
    val age = 18
    println("Hello")
    println("My name is $name")
    println("My age is $age")
    val a = 10
    val b = 15
    val c = a + b + 8
    val str = c.toString()
    println(str)
    val numList = arrayOf(1, 2, 3)
    var x = 0
    while (x < 3) {
        println("Item $x is ${numList[x]}")
        x = x + 1
    }
}

```

Шаг 8. Coding conventions

Мы разобрали новые конструкции, самое время узнать новые правила форматирования и кодирования.

- Имена переменных должны быть написаны в стиле CamelCase и начинаться со строчной буквы.
- При объявлении переменной после двоеточия при указании типа данных должен быть пробел.

space



```
val discount: Double = .20
```

- = Перед и после оператора, например, оператора присваивания (=), сложения (+), вычитания (-), умножения (*), деления (/) и других, должен быть пробел .

space
↓ ↓
var pet = "bird"

space
↓ ↓
val sum = 1 + 2

- При написании более сложных программ рекомендуется ограничить длину строки 100 символами. Это гарантирует, что вы сможете легко прочитать весь код программы на экране компьютера, без необходимости горизонтальной прокрутки при чтении кода.

Для автоформатирования можно нажать сочетание клавиш **Ctrl+Alt+L**.

Шаг 9. Преобразование данных

В Kotlin часто требуется преобразовывать значения одного типа в другой. Для этого у базовых типов (Int, Double, Long и др.) есть специальные методы:

- **toByte(), toShort(), toInt(), toLong()**
- **toFloat(), toDouble(), toChar(), toString()**

Пример преобразования:

```
fun main(){  
    val number: Double = 12.7  
    val integerNumber: Int = number.toInt() // Преобразует Double в Int  
        (отбрасывает дробную часть)  
    println(integerNumber) // Выведет: 12  
}
```

Задание:

1. Создайте переменные a: Int = 7 и b: Double = 3.5.
2. Преобразуйте b в Int и выполните операции:
 - Сложение a + b
 - Деление a / b
3. Выведите результаты в консоль.

Шаг 10. Поиск ошибок: работа с IDE

Ниже представлен код, который не компилируется. Ваша задача — найти и исправить все ошибки (с помощью IntelliJ IDEA):

```
fun main() {
    var x: Int = 65.2
    var isPunk = true
    var message = 'Hello'
    isPunk = 10
    var y = 15
    y = y + "hello"
}
```

Цель: научиться пользоваться подсказками IDE и исправлять базовые синтаксические ошибки.

Шаг 11. Консольный ввод и вывод

В Kotlin для работы с консолью используются:

1. Вывод данных:

- **print()** — выводит текст без перевода строки.
- **println()** — выводит текст и переходит на новую строку.

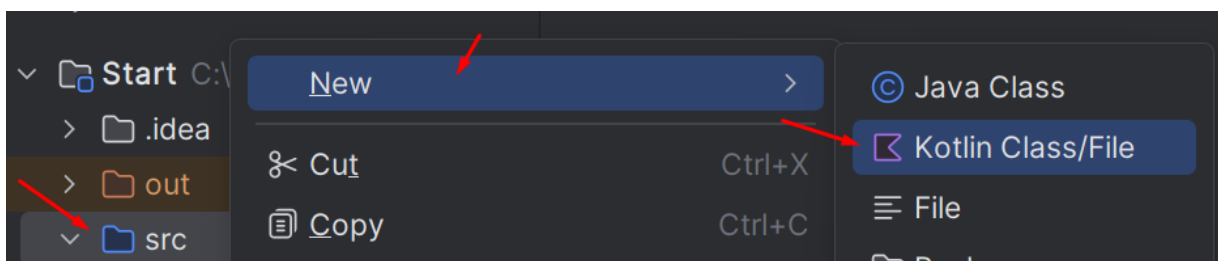
Пример:

```
fun main(){
    print("Hello, ")
    println("Kotlin!")
}
```

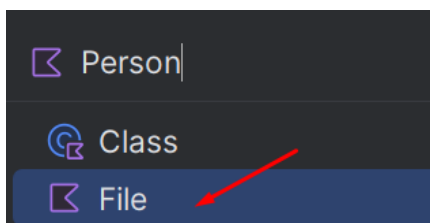
2. Ввод данных:

Для ввода данных используется функция **readln()**

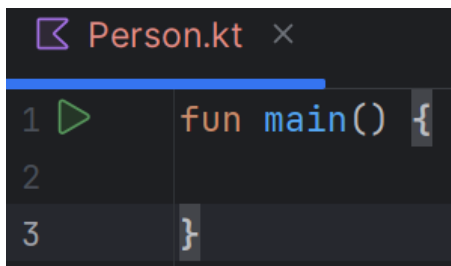
Давайте создадим в папке **src** новый файл:



Имя **Person**:

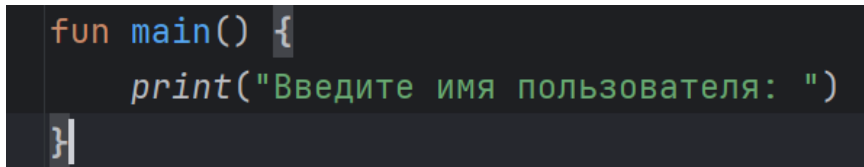


Добавьте точку входа main:



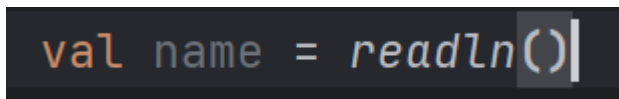
```
Person.kt x
1 fun main() {
2
3 }
```

Давайте создадим простую программу, которая запрашивает имя пользователя, а после выводит его на консоль. В начале спросим имя у пользователя:



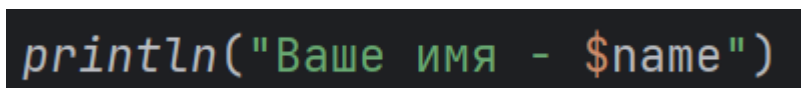
```
fun main() {
    print("Введите имя пользователя: ")
}
```

Затем считаем его и сохраним в консоль:



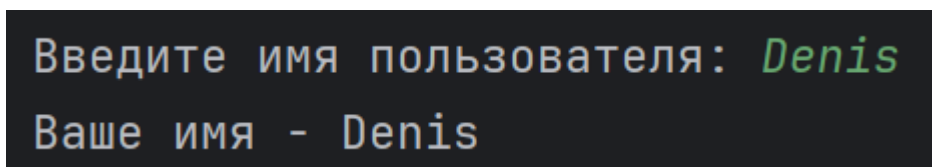
```
val name = readln()
```

И после выведем на консоль:



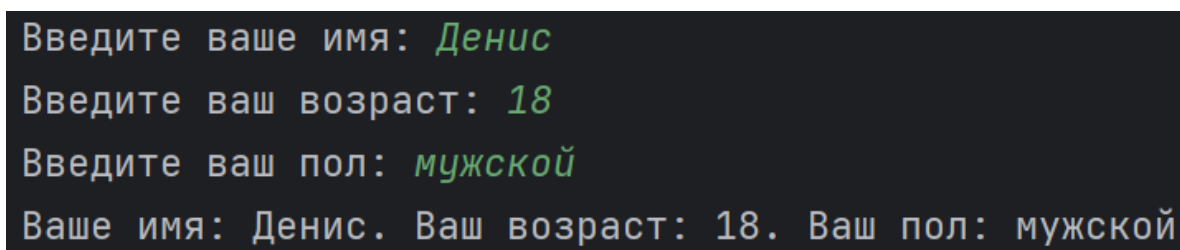
```
println("Ваше имя - $name")
```

Запустите программу, и убедитесь в корректной работе:



```
Введите имя пользователя: Denis
Ваше имя - Denis
```

Теперь усложните программу, запрашивайте не только имя пользователя, но его возраст и пол, и после выводило на консоль:



```
Введите ваше имя: Денис
Введите ваш возраст: 18
Введите ваш пол: мужской
Ваше имя: Денис. Ваш возраст: 18. Ваш пол: мужской
```

Задание:

1. Спросите у пользователя:
 - имя
 - возраст
2. Преобразуйте возраст в Int и выведите сообщение:
 - *"Через 5 лет вам будет X лет."*

```
Введите ваше имя: Денис
Введите ваш возраст: 18
Через 5 лет вам будет 23 лет.
```

Шаг 12. Базовые математические операции с целыми числами

В чем разница между Int переменной со значением 2 и String переменной со значением "2"? Когда они обе выводятся на печать, они выглядят одинаково.

Преимущество хранения целых чисел в виде Int (в отличие от String) заключается в том, что вы можете выполнять математические операции с Int переменными, такие как сложение, вычитание, деление и умножение. Например, две целые переменные можно сложить, чтобы получить их сумму.

1. Создайте новую программу, в которой вы определяете целочисленную переменную для количества непрочитанных писем в почтовом ящике и инициализируете ее значением 5. Определите вторую целочисленную переменную для количества прочитанных писем в почтовом ящике. Инициализируйте ее значением 100. Затем выведите общее количество сообщений в почтовом ящике, сложив два целых числа:

```
fun main() {
    val unreadCount = 5
    val readCount = 100
    println("У вас ${unreadCount + readCount} сообщений.")
}
```

2. Запустите программу, и она должна отобразить общее количество сообщений в почтовом ящике:

```
У вас 105 всего сообщений.
```

Для шаблона строки вы узнали, что можно поместить \$ символ перед одним именем переменной. Однако, если у вас более сложное выражение, вы должны заключить выражение в фигурные скобки с \$символом перед фигурными скобками: \${unreadCount + readCount}. Выражение внутри фигурных скобок, unreadCount + readCount, вычисляется как 105. Затем значение 105 подставляется в строковый литерал.

expression	value
<code>unreadCount + readCount</code>	105

Реализуйте следующие операции:

```
fun main(){
    val a = 10
    val b = 3
    println("Сумма: ${a + b}")
    println("Разность: ${a - b}")
    println("Произведение: ${a * b}")
    println("Частное: ${a / b}")
    println("Остаток: ${a % b}")
}
```

Задание:

1. Напишите программу, которая запрашивает у пользователя два числа.
2. Выполните над ними все вышеуказанные операции.

Шаг 13. Инкремент и декремент

Создайте следующую программу:

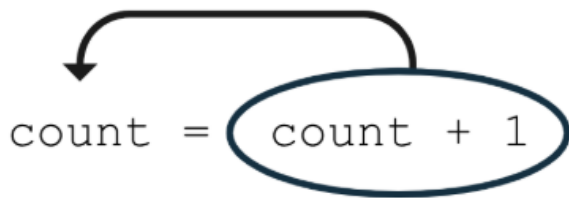
```
fun main() {
    var count: Int = 10
    println(count)
}
```

Допустим в своей программе мы хотим увеличить значение **count** на 1.

Для этого допишите ваш код:

```
fun main() {
    var count: Int = 10
    count = count + 1
    println(count)
}
```

Выражение справа от знака равенства равно `count + 1` и вычисляется как 11. Это потому, что текущее значение `count` равно 10 (которое находится в строке 2 программы) и `10 + 1` равно 11. Затем с помощью оператора присваивания значение 11 присваивается или сохраняется в `count` переменной.



Для краткости, если вы хотите увеличить переменную на 1, вы можете использовать *оператор инкремента* (++), который состоит из двух символов плюс. Используя эти символы непосредственно после имени переменной, вы сообщаете компилятору, что хотите прибавить 1 к текущему значению переменной, а затем сохранить новое значение в переменной. Следующие две строки кода эквивалентны, но использование ++ оператора приращения требует меньшего набора текста:

```
fun main() {  
    var count: Int = 10  
    count++  
    println(count)  
}
```

Также есть оператор декремента, который уменьшает значение переменной на 1. Он состоит из двух символов минуса:

```
fun main() {  
    var count: Int = 10  
    count++  
    count--  
    println(count)  
}
```

Шаг 14. Целочисленное деление

В Kotlin есть два основных оператора для работы с целыми числами:

1. / — обычное деление (возвращает **Int**, если оба числа Int, иначе Double)
2. % — остаток от деления (модуль)

Перепишите и запустите следующую программу:

```
val a = 5  
val b = 2  
println(a / b) // 2 (целая часть)  
println(a % b) // 1 (остаток)
```

Давайте решим задачу конвертации числа в часы, минуты и секунды.

Условие: Дано общее количество секунд (например, 7385). Нужно перевести в формат "ЧЧ:ММ:СС".

В начале мы должны запросить у пользователя число и сохранить в переменную:

```
val totalSeconds = readln().toInt()
```

Мы знаем, что в одном часе 60 минут. В одной минуте 60 секунд. Следовательно, в одном часе = $60 * 60 = 3600$ секунд.

Поэтому найдём количество часов поделив нашу переменную на 3600:

```
val hours = totalSeconds / 3600 // 1 час = 3600 секунд
```

Теперь нам нужно найти от исходного числа количество минут, для этого мы находим сперва остаток от деления на 3600, а после выполняем целочисленное деление на 60:

```
val minutes = totalSeconds % 3600 / 60 // 1 минута = 60 секунд
```

И последнее, это нужно найти секунды. Для этого мы должны найти остаток от деления на 3600 и остаток от деления на 60:

```
val seconds = totalSeconds % 3600 % 60
```

И в конечном итоге выводим на консоль наше время:

```
println("$hours ч $minutes мин $seconds сек")
```

Запустите программу и проверьте что всё работает корректно.

Шаг 15. Дробные числа

Kotlin поддерживает два типа данных для хранения **дробных чисел**:

Тип	Размер	Пример использования
Float	32 бита	val x: Float = 1.5f
Double	64 бита	val y: Double = 3.1415

По умолчанию дробные литералы — **Double**. Чтобы явно задать **Float**, нужно добавить f или F в конце: 3.14f

Давайте рассмотрим классический пример перевод температуры из фаренгейта в цельсии. Пользователь вводит число и его нужно перевести по формуле:

$$C = \frac{5}{9}(F - 32)$$

Очевидно, что температуры могут быть дробными, поэтому нам нужно при считывании с консоли преобразовать в тип Double:

```
val fahrenheit = readln().toDouble()
```

Затем мы выполняем по формуле перевод и сохраняем в переменную:

```
val celsius = (fahrenheit - 32) * (5 / 9)
```

После выводим значение на экран:

```
println("$fahrenheit°F = $celsius°C")
```

Для вставки спецсимвола можно зажать **Alt и ввести на **Numpad** 0176*

Введите значение 100, и попробуйте сказать корректный ли результат мы получили.

Самостоятельные задания

Задание 1. Переменные

Создайте следующие переменные и выведите их на консоль:

- Ваша любимая игра/кино/аниме;
- Ваша любимая цифра;
- Значение числа пи;
- Ваша любимая буква алфавита;

Задание 2. Работа с IDE

Намеренно допустите 3 ошибки в коде (например, удалите), ;, кавычки), исправьте их с помощью подсказок IDE.

Задание 3. Вычисление площади круга

Объявите переменные:

- val pi = 3.14 (Double)
- val radius = 5 (Int)

Выведите площадь круга (pi * radius * radius) без преобразования типов. Почему результат некорректный? Исправьте ошибку.

Задание 4. Консольный ввод/вывод

Запросите у пользователя имя и год рождения, выведите:

"[Имя], вам [2025 - год рождения] лет".

(используйте `readln()` и шаблоны строк \$).

Задание 5. Математика

Пользователь вводит два числа. Нужно вывести их сумму, разность и произведение в формате:

"число1 [операнд] число2 = результат".

Задание 6. Сумма чисел

Введите 3-значное число, выведите сумму его цифр (например, $123 \rightarrow 1+2+3 = 6$).

Задание 7. Кастомный разделитель

Напишите программу, которая считывает строку-разделитель и три строки, а затем выводит указанные строки через разделитель.

```
!=
1
2
3
1!=2!=3
```

Задание 8. Три последовательных числа

Напишите программу вывода на экран трех последовательно идущих чисел, каждое на отдельной строке. Первое число вводит пользователь, остальные числа вычисляются в программе.

```
12
12
13
14
```

Задание 9. Перевернутое число

Дано **трехзначное** число. Переверните число и выведите.

```
456
654
```

Задание 10. Число сотен числа

Дано **натуральное** целое число. Найдите число сотен(то есть третью справа цифру).

457789

7